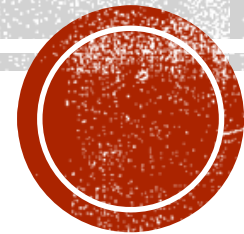


RECONFIGURABLE COMPUTING

CS G553: Reconfigurable computing

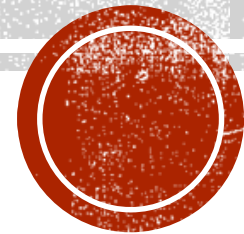
Dr. Meetha V Shenoy

Lect-10



HIGH LEVEL SYNTHESIS FOR RECONFIGURABLE DEVICES

COMPILATION & OPTIMIZATION



Control Dataflow Graphs

The sequencing graphs

Finite state machine with datapath



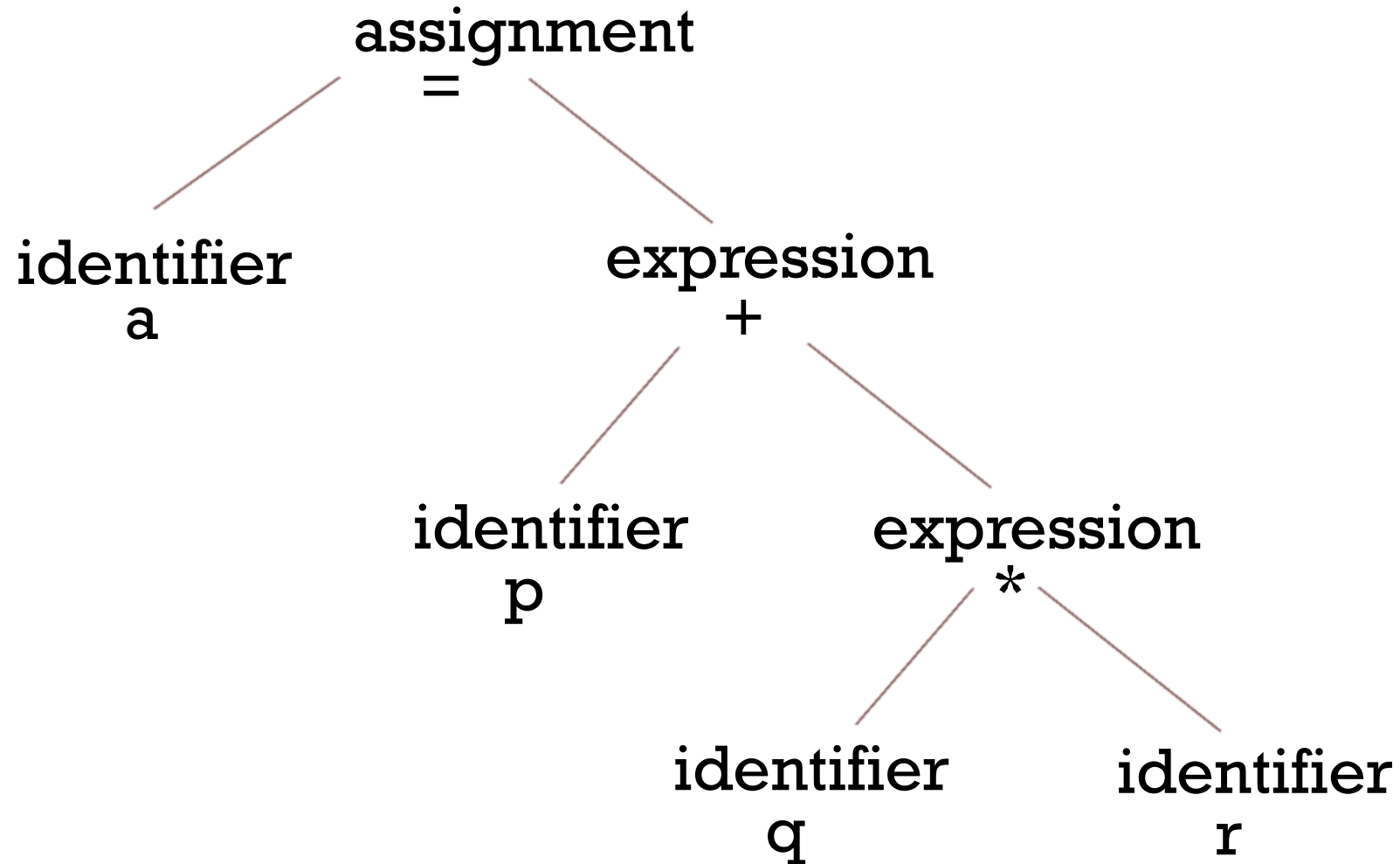
Lexical analysis breaks source code into meaningful units called tokens by scanning characters,

Parsing takes these tokens and builds a parse tree to check the code's grammatical structure and relationships between tokens



Compilation and Optimization

Parse Tree example $a = p + q * r$



Compilation and Optimization

Software Compilation

Compile program into intermediate form

Optimize intermediate form.

Generate target code for an architecture.



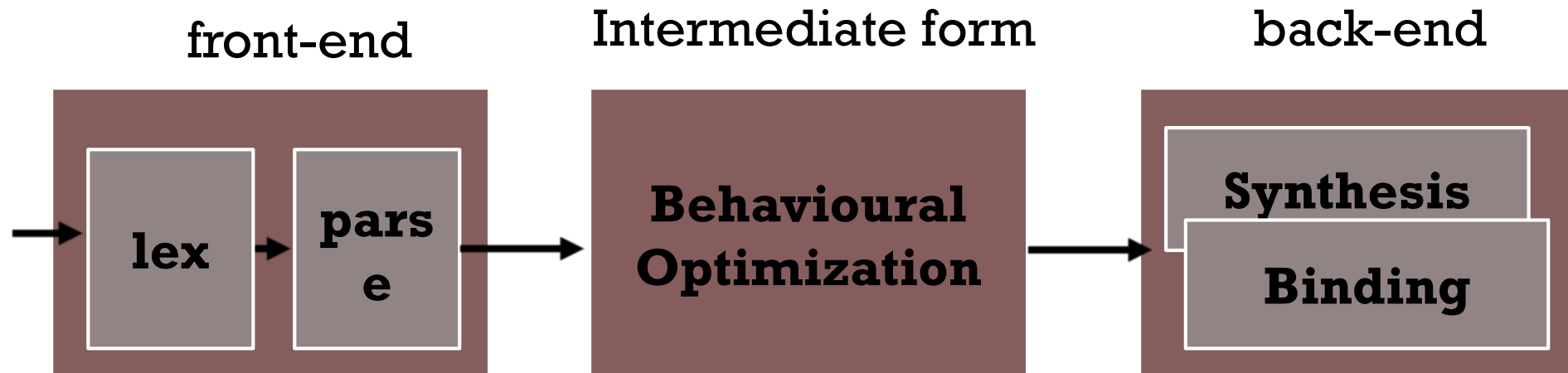
Compilation and Optimization

Hardware Compilation

Compile model into sequencing graph.

Optimize sequencing graph.

Generate gate-level interconnection for a cell library.



Compilation and Optimization

Front-end

Lexical and syntax analysis.

Parse-tree generation.

Macro-expansion.

Semantic analysis

Data-flow and control-flow (branch, loop, function) analysis.

Type checking.

Resolve arithmetic and relational operators.



Behavioral Level Optimization

Semantic-preserving transformations aiming at simplifying the model.

Applied to parse-trees or sequencing graphs during their generation.

Taxonomy

Data-flow based transformations.

Control-flow based transformations.



Data-flow based transformations

Tree-height reduction (arithmetic expression $\rightarrow$ parallelism).

Constant and variable propagation.

Common sub-expression elimination.

Dead-code elimination.

Operator-strength reduction.

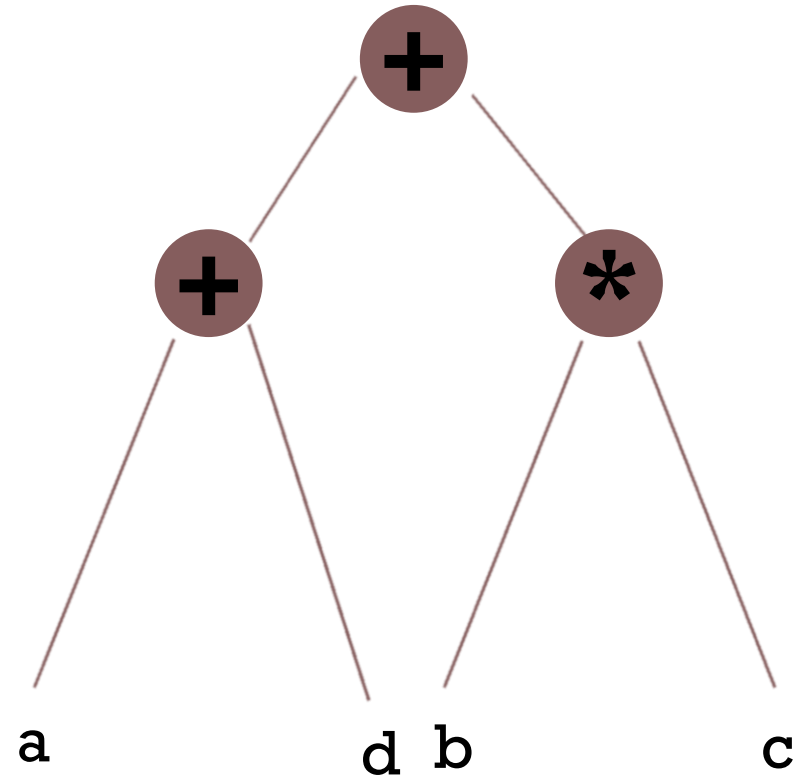
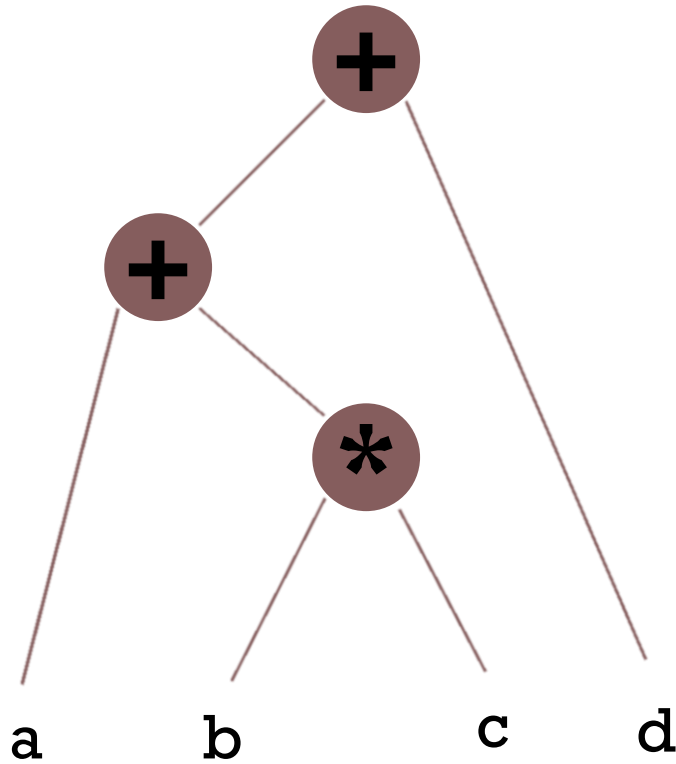
Code motion.



Tree-height reduction

Commutative and associative properties

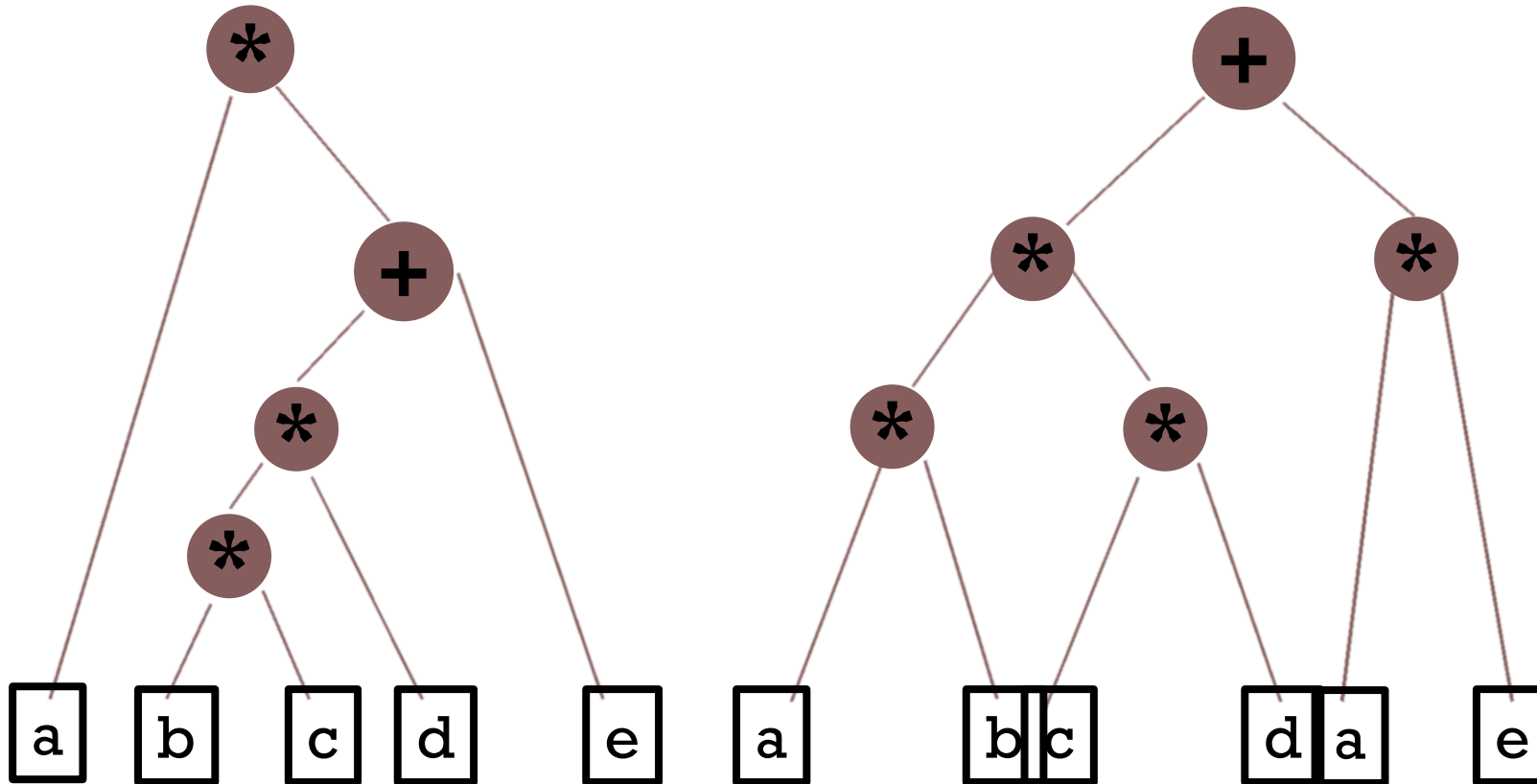
$$x = a + b * c + d \Rightarrow x = (a + d) + b * c$$



Tree-height reduction

Distributive property

$$x = a * (b * c * d + e) \Rightarrow x = a * b * c * d + a * e$$



Constant propagation

```
a = 0;  
b = a + 1;  
c = 2 * b;
```

```
a = 0;  
b = 1;  
c = 2;
```



Variable propagation

```
a = x;  
b = a+1;  
c = 2 * a;
```

```
a = x;  
b = x+1;  
c = 2 * x;
```



Subexpression elimination

Search isomorphic patterns in the parse trees.

```
a = x+y;  
b = a+l;  
c = x+y;
```

```
a = x+y;  
b = a+l;  
c = a;
```



Dead-code elimination

```
a = x;  
b = x+1;  
c = 2 * x;
```

`a = x;` can be removed if not referenced.



Operator-strength reduction

```
a = x2 ;  
b = 3 * x;
```

```
a = x * x;  
t = x << 1;  
b = x+t.
```



Code motion

```
for (i = 1; i < a * b)  
{ } ;
```

```
t = a * b;  
for (i = 1; i < t) { }.
```



Control-Flow Based Transformations

Model expansion.

Conditional expansion.

Loop expansion.

Block-level transformations.



Model Expansion

Expand subroutine → flatten hierarchy.

Useful to expand scope of other optimization techniques.

Problematic when routine is called more than once.

```
x = a+b; y = a * b; z = foo(x, y);  
foo(p, q){ t = q - p; return(t); }
```

```
x = a+b; y = a * b; z = y - x
```



Conditional expansion.

Transform conditional into parallel execution with test at the end.

Useful when test depends on late signals.

May preclude hardware sharing. Always useful for logic expressions.

```
y = ab;  
if (a) {x = b + d; } else {x = bd;}
```

```
y = ab;  
x = a (b+d) + a'bd
```

```
y = ab;  
x = y + d (a+b)
```

$ab+ad+a'bd$
 $Y+d(a+a'b)$
 $Y+d(a+a')(a+b)$
 $Y+d(a+b)$



Conditional expansion.

May preclude hardware sharing? Prevent hardware sharing:

If else being transformed into parallel execution of both the (TRUE and FALSE) path.

Suppose if else was introduced to share a resource, then conditional expansion prevents hardware sharing.



Loop Expansion

Applicable to loops with data-independent exit conditions.

Useful to expand scope of other optimization techniques.

Problematic when loop has many iterations.

```
x = 0;  
for (i = 1; i ≤ 3; i++)  
{x = x+1;}
```

```
x = 0;  
x = x+1; x = x+1; x = x+1
```



Loop Expansion

Applicable to loops with data-independent exit conditions.

Useful to expand scope of other optimization techniques.

Problematic when loop has many iterations.

```
x = 0;  
for (i = 1; i ≤ 3; i++)  
{x = x+i;}
```

```
x = 0;  
x = x+1; x = x+2; x = x+3
```

